

エラー処理設計書

1. エラー分類

1.1 エラーレベル

CRITICAL (致命的)

- システム全体が停止する可能性があるエラー
- 例: データベース接続失敗、設定ファイル読み込み失敗

ERROR (エラー)

- 特定のアカウント処理が失敗したエラー
- 例: ログイン失敗、アプリ停止検知、認証エラー

WARNING (警告)

- 処理は継続できるが、注意が必要な状態
- 例: タイムアウト、リトライ成功、手動操作検知

INFO (情報)

- 正常な処理の記録
- 例: 処理開始、処理完了、設定変更

1.2 エラータイプ

ネットワークエラー

- **タイプ:** 一時的エラー
- **原因:** 接続タイムアウト、DNS解決失敗、SSL証明書エラー
- **対応:** 自動リトライ (最大3回、指数バックオフ)

認証エラー

- **タイプ:** 致命的エラー (アカウント単位)
- **原因:** ログイン失敗、セッション期限切れ、アカウントロック
- **対応:** エラーログ記録、通知送信、該当アカウントスキップ

アプリ停止エラー

- **タイプ:** 致命的エラー (アカウント単位)
- **原因:** 「赤い×」検知、ブラウザクラッシュ
- **対応:** 自動再起動、セッション復元、処理再開

セッション競合エラー

- **タイプ:** 警告 (アカウント単位)
- **原因:** 手動操作検知、Session Token変化
- **対応:** 該当アカウントスキップ、ログ記録、他アカウント継続

DOM要素検索エラー

- **タイプ:** エラー (アカウント単位)
- **原因:** 要素が見つからない、ページ構造変更
- **対応:** スクリーンショット保存、エラーログ記録、通知送信

データベースエラー

- **タイプ:** 致命的エラー (システム全体)
- **原因:** 接続失敗、クエリエラー、トランザクション失敗
- **対応:** リトライ、フォールバック、システム停止

2. エラーハンドリング戦略

2.1 リトライ戦略

```
# 指数バックオフリトライ
def retry_with_backoff(func, max_retries=3, base_delay=1):
    for attempt in range(max_retries):
        try:
            return func()
        except RetryableError as e:
            if attempt == max_retries - 1:
                raise
            delay = base_delay * (2 ** attempt)
            time.sleep(delay)
```

2.2 サーキットブレーカーパターン

```
# アカウントごとのサーキットブレーカー
class AccountCircuitBreaker:
    def __init__(self, failure_threshold=5, timeout=300):
        self.failure_count = 0
        self.failure_threshold = failure_threshold
        self.timeout = timeout
        self.last_failure_time = None
        self.state = "CLOSED" # CLOSED, OPEN, HALF_OPEN
```

2.3 エラー通知戦略

通知タイミング

- **CRITICAL:** 即座に通知

- ERROR: 5分以内に3件以上発生した場合
- WARNING: 1時間に10件以上発生した場合

通知方法

- Chrome拡張機能経由のポップアップ
- ログファイルへの記録
- Web管理画面への表示

3. エラーログ設計

3.1 ログフォーマット

```
{
  "timestamp": "2024-12-20T10:30:45.123Z",
  "level": "ERROR",
  "account_id": "account_001",
  "account_name": "店舗A",
  "error_type": "AUTHENTICATION_ERROR",
  "error_code": "LOGIN_FAILED",
  "message": "ログインに失敗しました",
  "details": {
    "url": "https://doors1.shinchakun.info/dokodemo/#/",
    "retry_count": 3,
    "stack_trace": "..."
  },
  "context": {
    "task_type": "SCHEDULE_UPDATE",
    "session_id": "session_12345"
  }
}
```

3.2 ログ保存先

- ファイルログ: logs/app_YYYY-MM-DD.log
- エラーログ: logs/error_YYYY-MM-DD.log
- データベース: error_logsテーブル

3.3 ログローテーション

- ファイルサイズ: 10MB
- バックアップ数: 5ファイル
- 保存期間: 30日間

4. エラー復旧戦略

4.1 自動復旧

アプリ停止

1. 「赤い×」検知
2. ブラウザセッション終了
3. 新しいセッション開始
4. ログイン処理
5. 処理再開

セッション期限切れ

1. セッション期限切れ検知
2. 自動再ログイン
3. 処理継続

ネットワークエラー

1. タイムアウト検知
2. 指数バックオフでリトライ
3. 成功したら処理継続

4.2 手動復旧

修正パッチ適用

1. パッチファイル配置
2. システム再起動
3. パッチ自動適用
4. 動作確認

設定変更

1. Web管理画面で設定変更
2. 設定ファイル更新
3. サービス再起動

5. エラー監視

5.1 リアルタイム監視

- Web管理画面でのエラー一覧表示
- エラー発生時の即座通知
- エラー統計の可視化

5.2 定期レポート

- 日次エラーレポート
- 週次エラー統計
- 月次エラー分析

6. エラー修正マニュアル

6.1 よくあるエラーと対処法

ログイン失敗

1. パスワード確認
2. アカウントロック確認
3. URL確認 (doors1/doors2)
4. 手動ログインテスト

アプリ停止

1. ブラウザバージョン確認
2. Playwrightバージョン確認
3. システムリソース確認
4. ログ確認

セッション競合

1. 手動操作の有無確認
2. セッション管理設定確認
3. タイミング調整

6.2 パッチ適用手順

1. パッチファイルをpatches/ディレクトリに配置
2. システムを停止
3. パッチを適用 (自動または手動)
4. システムを再起動
5. 動作確認